

The image shows the exterior of a building with a large mural on the left wall. The mural features the UFCA logo at the top, followed by the text 'UNIVERSIDADE FEDERAL DO CARIRI' in bold black letters, and a large, stylized 'RUN' in orange and blue below it. At the bottom of the mural, the text 'UFCA RUN' is written in white. The building itself has a series of white columns and a covered walkway with a metal railing. The sky is blue with some clouds.

UFCA
UNIVERSIDADE
FEDERAL DO CARIRI

UFCA RUN

A ideia

Gênero: **endless runner** — referências Subway Surfers, Temple Run

O jogador é um estudante correndo pelos corredores da UFCA

O perseguidor é o professor

Objetivo: desviar de obstáculos, coletar provas, sobreviver o máximo possível

Por que um corredor

| **Corredor = geometria repetível: um chão e duas paredes**

Tempo economizado em modelagem foi para **iluminação, materiais e sombra**

Cena sempre em movimento → a iluminação dinâmica fica visível o tempo todo

Câmera fixa é justificada pelo gênero, não é limitação

Visão geral técnica

C++17 · OpenGL (pipeline fixo) · FreeGLUT

~1250 linhas em main.cpp + 3 headers escritos pela equipe

Única dependência externa: stb_image.h (decodifica PNG/JPEG)

Compila em Linux e Windows a partir do mesmo código



Roteiro da demonstração

1	Menu inicial	quatro opções, tudo em projeção ortogonal
2	Skins do professor (← →)	os quatro "professores"
3	Uma partida	troca de faixa, pulo, coleta de prova
4	A luz do teto passando	
5	A sombra esticando	
6	Game over de propósito	jumpscare do professor + screen shake
7	Créditos	encerra a demo

A/D ou setas trocam de faixa · W/Espaço pula · F tela cheia · R reinicia · ESC volta ao menu

Elementos de jogabilidade



3 faixas de corrida · pulo · 3 tipos de obstáculo

carteira

1.00

lixeira

0.81

livros

0.45

Provas espalhadas valem pontos

Perder: bater e ser alcançado

Progressão de dificuldade

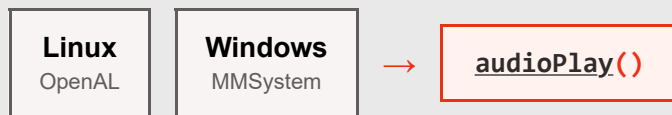
Velocidade cresce continuamente com o tempo

```
score += 10 * speedMultiplier
```

O mesmo multiplicador escala a pontuação

Recorde da sessão guardado e exibido no HUD

Áudio e portabilidade



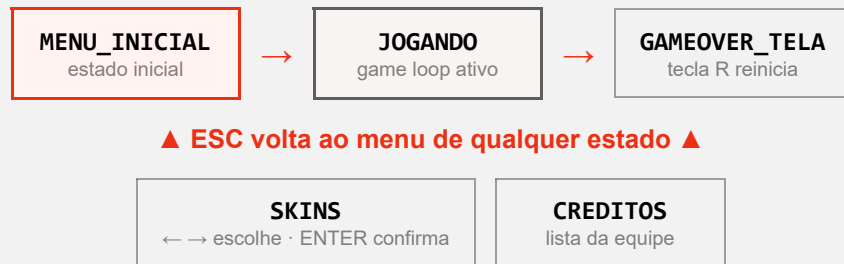
Sons de coleta e de colisão

make NO_AUDIO=1 compila sem som

Máquina de estados

5 estados num único enum GameState

O mesmo estado controla o que é desenhado e como o teclado responde



ENTER inicia o jogo no menu e confirma a skin na tela de skins — mesma tecla, comportamentos diferentes, um único enum.

Anatomia de um frame

`glutTimerFunc(16, ...)` → o loop se reagenda a cada 16 ms ≈ 60 FPS

1. <code>glClear</code>	zera cor e Z-buffer do frame anterior
2. <code>gluLookAt</code>	posiciona a câmera na modelview
3. <code>atualizarSpotsTeto()</code>	luzes enviadas DEPOIS da câmera
4. <code>desenharCorredores()</code>	chão e paredes, 45 segmentos cada
5. <code>desenharCena()</code>	professor · jogador · sombras · obstáculos · provas
6. <code>desenharHUD()</code>	troca para ortogonal, desliga depth e cull
7. <code>glutSwapBuffers()</code>	o buffer pronto vai para a tela de uma vez

Movimentação entre faixas

3 faixas fixas em X

A seta **não teleporta** o jogador: altera um `targetPlayerX`

A posição real caminha até o alvo alguns por cento por frame

Pulo: parábola com gravidade

Colisão

Colisão = mesma faixa e distância em Z dentro de uma janela

Hitbox derivada do modelo: `PLAYER_COLLISION_Z = playerZ + getCenter().z`



Punição tem caixa apertada, recompensa tem caixa folgada. Ajuste de sensação, não de física.

O formato .obj

.obj é texto puro — cada linha começa com uma tag
v vértice · vt coord. de textura · vn normal · f face

```
# Blender 5.2.0 LTS  
  
mtllib estudanteGraduando.mtl  ← qual arquivo de materiais usar  
  
v 0.016 0.821 3.726  ← um vértice (x y z)  
vt 0.512 0.338  ← coordenada de textura (u v)  
usemtl Black  ← material "Black" a partir daqui  
f 1/1/1 2/2/2 22/3/3  ← face: índices v/vt/vn
```

Os índices começam em 1, não em 0 — primeira fonte de bug do parser.

O que o carregador entrega

Listas de vértices, normais e coordenadas de textura

Lista de faces, cada uma com o **índice do material** que a pinta

Bounding box → altura do obstáculo e hitbox do jogador

Fallback: se o .obj falhar, desenha um cubo em vez de quebrar

estudante

carteira

lixeira

livros

prova

Do .mtl para o OpenGL

Ka	→	GL_AMBIENT	=	quanto reflete da luz ambiente
Kd	→	GL_DIFFUSE	=	a cor própria sob luz direta
Ks	→	GL_SPECULAR	=	a cor do brilho refletido
Ns	→	GL_SHININESS	=	concentração do brilho — não a intensidade

aplicarMaterial() faz exatamente estas quatro chamadas, uma vez por grupo usemtl.

Um objeto, vários materiais

`drawWithMaterials()` percorrendo a carteira universitária

`usemtl metal_estrutura`

Ns 260 · reflexo concentrado

→ desenha as faces da estrutura

`usemtl plastico_assento`

Ns 70 · brilho médio

→ desenha as faces do assento

`usemtl madeira_prancheta`

Ns 15 · praticamente fosco

→ desenha as faces da prancheta

Uma única chamada de desenho, três acabamentos ópticos diferentes no mesmo objeto.

Cores distintas na prática

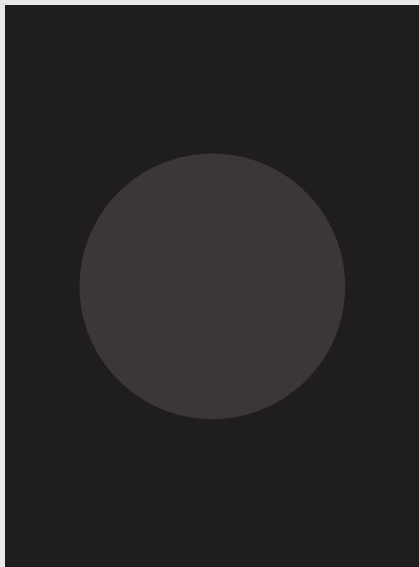
MODELO	MATERIAIS	CONTRASTE
livros.obj	5	4 capas coloridas + páginas (Ns 90 vs 10)
carteira.obj	3	metal 260 · plástico 70 · madeira 15
lixeira.obj	3	plástico verde · aro metálico 240 · saco preto
prova.obj	2	papel fosco 10 · nota dourada 200
estudanteGraduando.obj	5	pele · capelo preto · borlas Ns 250



As três componentes

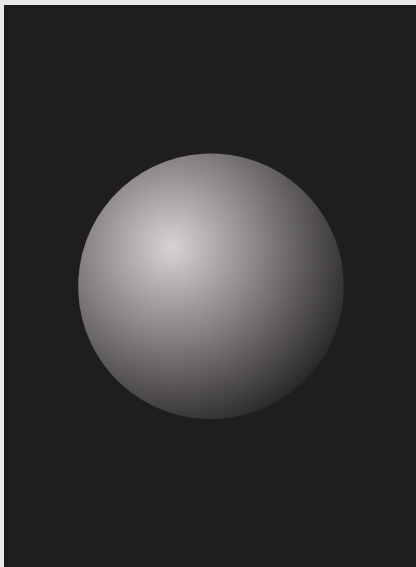
Ambiente

piso mínimo de luz



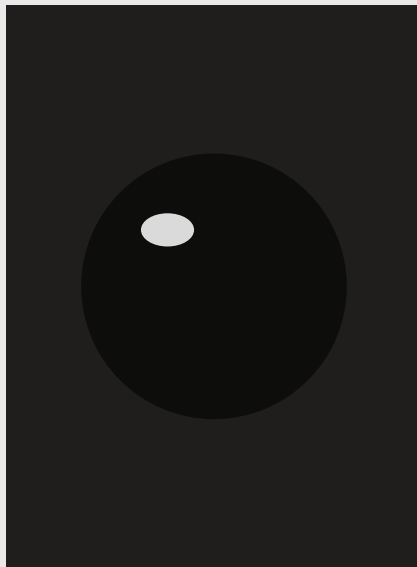
Difusa

depende do ângulo luz × normal



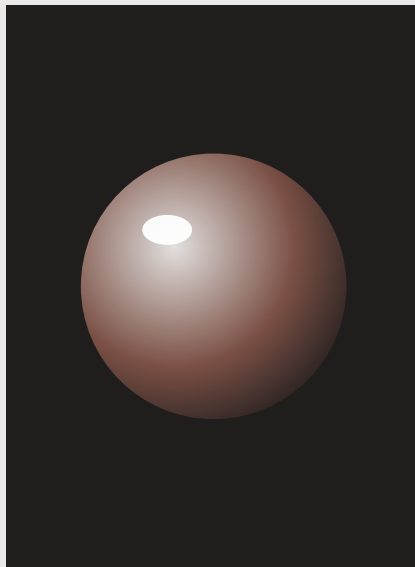
Especular

brilho ao se aproximar



Soma

resultado final



Ambiente da cena: `GL_LIGHT_MODEL_AMBIENT`

Ideia

Demo

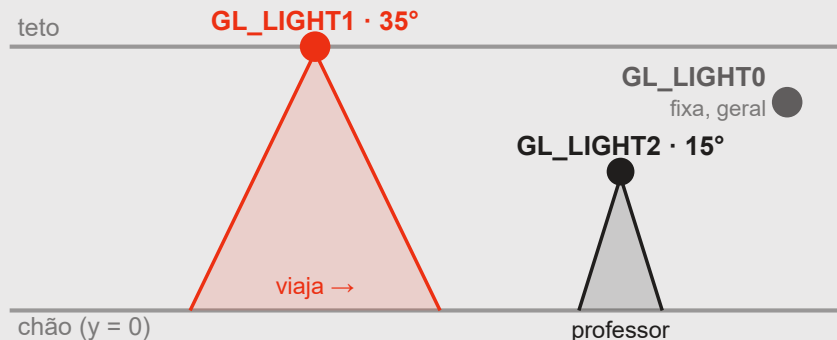
Lógica

Modelagem

Luz e sombra

Câmera

Três fontes de luz



GL_LIGHT0 — fixa, geral, três componentes definidas separadamente

GL_LIGHT1 — spot que viaja pelo teto, cone 35°, expoente 10

GL_LIGHT2 — spot fechado, cone 15° e expoente 60, segue o professor

Posição e direção reenviadas a cada frame

Por que a luz vem depois da câmera

glLight ANTES de gluLookAt

A luz gruda na câmera e viaja junto com ela

✗ o corredor nunca escurecia

glLight DEPOIS de gluLookAt

A luz fica ancorada em coordenadas de mundo

✓ o efeito de varredura aparece

A posição de uma luz é transformada pela modelview no momento em que é enviada — foi um bug real até a ordem ser corrigida.

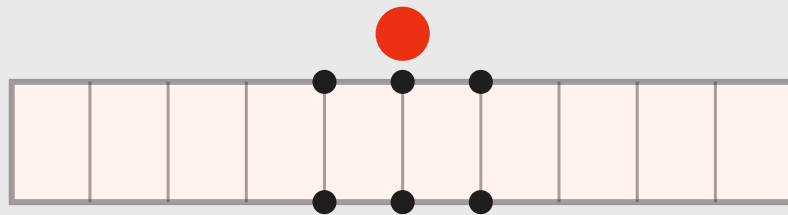
Por que o corredor tem 45 segmentos

✗ Um quad único

✓ 45 segmentos



A luz cai no meio, mas os 4 vértices estão longe — a face inteira fica escura.

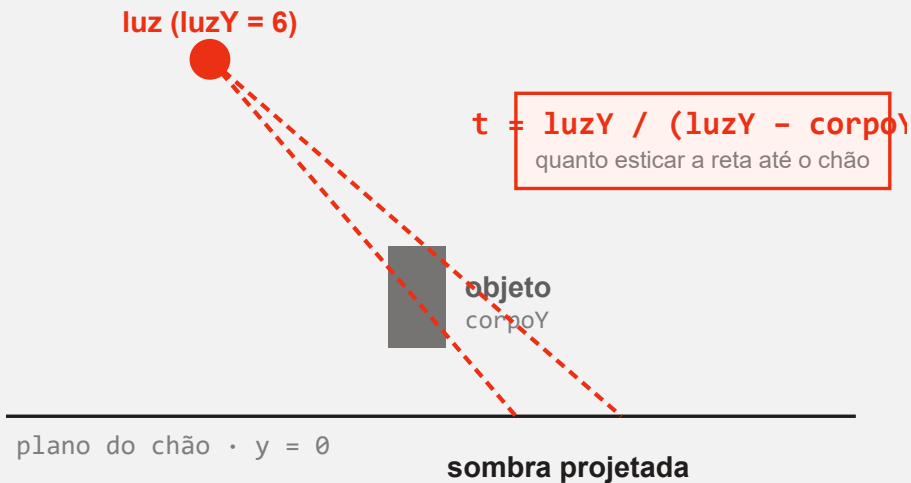


Há vértices sob a luz — recebem iluminação e ela é interpolada pelas faces vizinhas.

Iluminação calculada por vértice, interpolada (Gouraud)

Subdividir a geometria é o que torna a luz móvel visível

Projeção de sombra



O próprio modelo é achatado: `glScalef(escala, 0.01f, escala)`

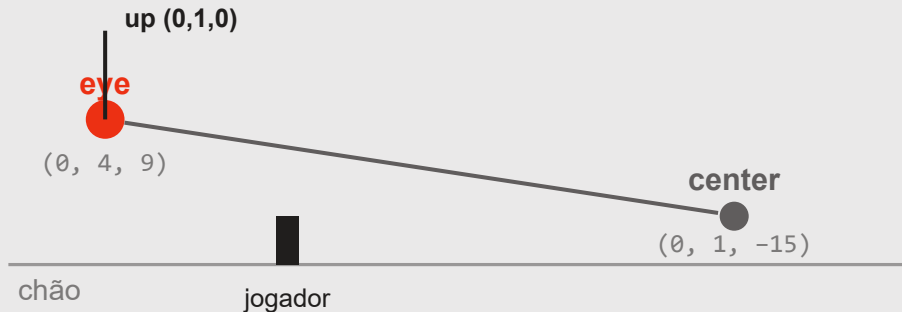
Opacidade cai com a distância da luz, mas nunca chega a zero

GL_BLEND ligado, GL_LIGHTING desligado

Mesma função para o jogador e para os obstáculos

A câmera

`gluLookAt(0,4,9 → 0,1,-15 → up 0,1,0)`



Terceira pessoa, atrás e acima do jogador A câmera não olha para o jogador: olha para o fundo do corredor

A câmera

Na colisão, uma translação aleatória é aplicada sobre a matriz da câmera

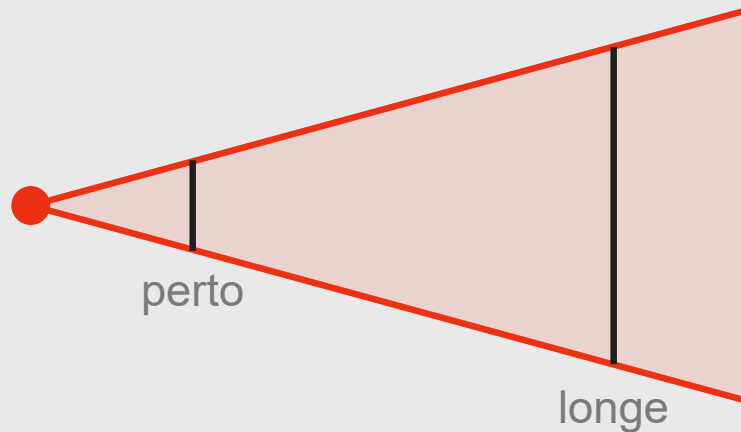
Como vem depois do `gluLookAt`, a cena inteira treme junta

Dura poucos frames e some sozinha

Num endless runner a leitura da pista precisa ser constante, o enunciado pede câmera para navegação ou visualização.

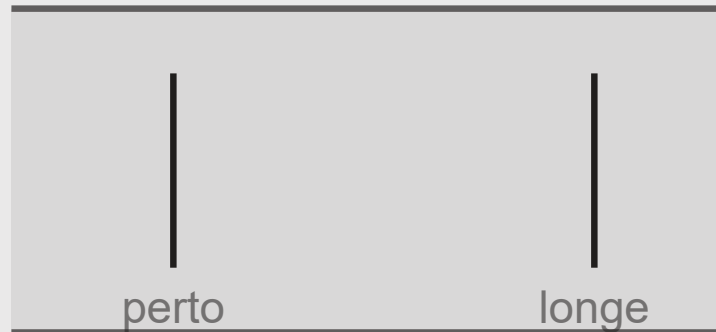
Duas projeções no mesmo frame

gluPerspective — a cena 3D



Frustum: o mesmo objeto ocupa menos tela quanto mais longe. FOV 70° exagera a profundidade e reforça a velocidade.

gluOrtho2D — o HUD



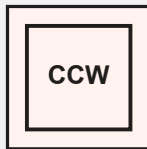
Caixa: o tamanho não muda com a distância. Coordenada vira pixel, origem no canto superior esquerdo.

Troca no meio do frame com `glPushMatrix/glPopMatrix`; aspect recalculado no `reshape()`.

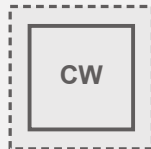
Remoção de superfícies ocultas

Z-buffer — `GL_DEPTH_TEST`, correção por pixel

Back-face culling — `GL_CULL_FACE`, otimização por face antes de rasterizar



Face da frente — é desenhada



Face de trás — é descartada

`glFrontFace(GL_CCW)`: sentido anti-horário na tela = frente. É por isso que a ordem dos vértices importa.

Dois bugs de orientação

1. A parede direita sumia

As duas paredes são espelhadas

Com a mesma ordem de vértices, a face "da frente" da direita apontava para fora

Correção: inverter a ordem de vértices só da parede direita

2. O menu inteiro sumia

gluOrtho2D monta o eixo Y invertido (origem no topo)

Inverter um eixo inverte a orientação de todos os polígonos

Correção: desligar o culling ao desenhar o HUD

Requisitos atendidos

REQUISITO	COMO FOI ATENDIDO	PARTE
Objetos 3D com cores distintas		4
Materiais / propriedades ópticas		4
Iluminação ambiente		5
Iluminação difusa		5
Iluminação especular		5
Câmera		6
Projeções		6
Remoção de superfícies ocultas		6
Extra · importação de objeto		4
Extra · texturas		1 e 5
Extra · projeção de sombra		5

Obrigado - Perguntas?